

Mobile Threat Defense (MTD) & Cloud Sandboxing Ecosystem

Complete End-to-End Architectural Walkthrough, Dataflow Pipelines, FinTech Heuristics, and Source Code Master Guide

Executive Overview: CipherScan is an enterprise-grade cybersecurity architecture engineered to intercept, sandbox, analyze, and neutralize zero-day web phishing, mobile malware, and fraudulent payment links on mobile endpoints *before any user exposure*. By pairing native Android OS Intent Interception with headless cloud Chromium sandboxing, external threat intelligence aggregation, and Indian FinTech heuristics, CipherScan provides comprehensive zero-trust link verification.

Document Classification

Engineering Architecture & Defense Master Guide

Target Audience

Computer Science Students, Evaluators & Software Engineers

Core Technology Stack

Android Kotlin, Node.js, Express, Playwright, Neon Postgres, React

Document Version & Status

v2.0 Production-Ready Release

1. Executive Summary & Layman Explanation

To understand CipherScan, let us start with the core security problem faced by every modern smartphone user.

What Happens When You Click a Link Today?

When an SMS arrives saying *"Your electricity bill is overdue, pay here: bit.ly/3x..."* or someone forwards a WhatsApp link promising a *"₹10,000 lottery cash prize"*, tapping that link immediately opens Google Chrome or Safari. The browser connects to the attacker's server, executes their scripts, downloads malicious payloads, or renders a convincing replica of a banking login page. **By the time you realize something is wrong, your phone and personal data are already compromised.**

The "Digital Bomb Squad" Analogy

CipherScan functions exactly like a digital bomb-defusal squad for links:

1 On-Device Interception

Instead of letting Chrome open the link, CipherScan's native Android engine catches the link the exact millisecond you tap it, freezing the navigation before the browser can download a single byte.

2 Cloud Sandboxing & Photograph

The link is sent to a remote cloud sandbox container running Headless Chromium. The cloud opens the website in complete isolation, follows all redirect hops (e.g., short links like Bitly), takes a real photograph (screenshot), and queries 70+ global antivirus engines.

3 Transparent Security Overlay

CipherScan displays a lightweight security card on your phone showing a 0–100 Risk Score, the unrolled destination URL, detected threats, and the actual preview screenshot so you can see what the page looks like without exposing your device.

4 Safe Hand-off Without Loops

If the link is safe, or if you explicitly choose to proceed, CipherScan cleanly hands the link to Google Chrome Custom Tabs (CCT) using dedicated package routing, ensuring the phone never gets trapped in an infinite loop.

2. High-Level Monorepo Architecture

CipherScan is engineered as an enterprise-grade **polyglot monorepo**. It integrates native mobile code, a cloud microservice API, a serverless proxy layer, a relational database, and an administrative web console.

Module / Directory	Technology Stack	Role & Key Responsibilities
android/	Kotlin, Android SDK (API 26–34), Retrofit, Coil, Material 3	Native Android client. Catches OS-level ACTION_VIEW intents for http , https , and upi , renders security overlays, and handles Chrome Custom Tabs dispatch.
artifacts/api-server/	Node.js, Express, TypeScript, Playwright Chromium, Pino	Cloud Threat Intelligence Engine. Performs SSRF-safe URL resolution, headless multi-hop sandboxing, threat feed queries, and Indian FinTech fraud heuristics.
artifacts/cipherscan/	React 18, Vite, Tailwind CSS, Recharts, Lucide, Vercel Serverless	Security Command Center. Features telemetry metrics, interactive manual URL/UPI analysis, paginated scan history, and a serverless Backend-For-Frontend (BFF) proxy.
lib/db/	Neon Serverless Postgres, Drizzle ORM, pg	Persistence layer storing comprehensive telemetry: original URLs, terminal URLs, risk scores, redirect hops, threat reasons, and Base64 previews.
lib/api-zod/	TypeScript, Zod	Runtime schema validation contracts shared across backend and frontend to prevent contract drift and serialization bugs.
lib/api-client-react/	TanStack React Query, Orval, Axios	Auto-generated type-safe React Query hooks connecting the web dashboard to backend API routes.

3. The 3-Gate Threat Defense Pipeline

To eliminate sluggish loading times without sacrificing security, CipherScan splits threat analysis into a high-performance **Tri-Gate Pipeline**:

Gate 1: Edge-First On-Device Interceptor (0ms Latency)

Before making any network requests, the Android client checks the clicked domain against a built-in zero-latency trust list (e.g., `google.com`, `wikipedia.org`, `github.com`). If a domain is globally trusted, it passes in **0 milliseconds** straight to Chrome Custom Tabs, saving cellular bandwidth, battery life, and cloud API quota.

Gate 2: Lightweight Redirect Stream Engine (Cloud Sandboxing)

For all uncached, shortened, or unfamiliar URLs, the engine spawns an isolated Chromium instance via Playwright:

- **Multi-Hop Unrolling:** Follows sequential HTTP 301/302 redirects (e.g. `bit.ly` → `tinyurl` → landing page) to expose the terminal domain.
- **Visual Evidence Capture:** Renders the landing page and captures a Base64 JPEG preview in an isolated cloud sandbox.
- **Reputation Aggregation:** Queries VirusTotal v3 (70+ antivirus vendors) and Google Safe Browsing v4 concurrently.

Gate 3: LegalTech & Indian FinTech Sentinel (UPI Guard)

Dedicated engine for payment links (`upi://pay`). Parses recipient Virtual Payment Address (VPA), transaction reference, and requested amount to flag fraudulent debit traps and fake customer care handles.

The Unified Risk Scoring Formula

CipherScan computes a normalized Risk Score between **0 and 100** using weighted heuristic factors:

```
Risk Score = (VirusTotal Hits * 25)
              + (Google Safe Browsing Flag ? 45 : 0)
              + (UPI Reverse-Debit Lure ? 60 : 0)
              + (Suspicious TLD / Entropy ? 20 : 0)
              + (High Redirect Depth ≥ 3 ? 15 : 0)
              // Clamped to maximum 100
```

Score Range	Verdict Badge	Action Taken
0 – 20	SAFE	Display green checkmark badge; user can proceed immediately to Chrome.
21 – 69	SUSPICIOUS	Yellow advisory overlay; displays screenshot preview and specific risk flags.
70 – 100	MALICIOUS	Red hard-block screen; prevents accidental taps; offers 1-tap Indian Cybercrime Dossier export.

4. Deep Dive: Android Mobile Client (`android/`)

1. Android Manifest Intent Interception

In standard Android development, an application can register to handle specific URI schemes by declaring `<intent-filter>` blocks in its `AndroidManifest.xml`. CipherScan registers for `http`, `https`, and `upi://pay` with the `BROWSABLE` category:

```
<activity
    android:name=".activity.LinkInterceptorActivity"
    android:theme="@style/Theme.CipherScan.Transparent"
    android:launchMode="singleTask"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="http" />
        <data android:scheme="https" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="upi" />
        <data android:host="pay" />
    </intent-filter>
</activity>
```

2. Transparent Gatekeeper: `LinkInterceptorActivity.kt`

- Configured with a fully transparent window theme, meaning no flickering window pops up when a link is tapped.
- Reads the raw incoming URI via `intent.dataString`.
- Shows a sleek, animated loading overlay with a spinning progress indicator.
- Dispatches an asynchronous Kotlin Coroutine using Retrofit to call `POST /api/analyze`.
- Once the response arrives, it instantiates `SecurityOverlayBottomSheet.kt`.

3. The "Loopback Evasion" Problem & Resolution

The Engineering Challenge: Infinite Intent Loops

If CipherScan is registered to handle `http` and `https` links, what happens when the user clicks "Proceed to Website"? If the app executed standard code like `startActivity(Intent(ACTION_VIEW, Uri.parse(url)))`, Android would evaluate all apps capable of opening web links, find CipherScan, and route the link back to CipherScan! The phone would be caught in an inescapable infinite loop.

The Solution in `BrowserLauncher.kt` :

1. CipherScan queries Android's `PackageManager` for all installed packages that handle web intents.
2. It explicitly excludes CipherScan's own package (`com.cipherscan.android`) from the list.
3. It selects the user's primary browser (e.g. `com.android.chrome`) and binds the launch intent directly to that package via `intent.setPackage(...)` or launches it via **Chrome Custom Tabs (CCT)**. This guarantees a safe handoff with zero loopbacks.

4. Custom UI & Hardware Monitors

- `RiskGaugeView.kt` : A custom Android `View` that draws a smooth, hardware-accelerated semicircular SVG arc from 0 to 100 with dynamic color transitions (Green → Amber → Red).
- `SecurityOverlayBottomSheet.kt` : A Material 3 BottomSheet dialog rendering the risk gauge, terminal URL, redirect hop count, coil-rendered screenshot lightbox, and action buttons.
- `ClipboardMonitor.kt` & `SmsReceiver.kt` : Background background services that watch the system clipboard and incoming SMS messages for newly copied or received links, providing proactive warnings.

5. Deep Dive: Cloud Threat Intelligence Engine

(`artifacts/api-server/`)

1. Core Request Orchestration: `analyze.ts`

When the Android app or Web Dashboard sends a URL to `POST /api/analyze`, the API server executes a strict, multi-step pipeline:

A Request Validation & Sanitization

The incoming JSON payload is validated using Zod. Unsanitized protocols, null bytes, and malformed characters are rejected before any processing occurs.

B Server-Side Request Forgery (SSRF) Guard (`urlSafety.ts`)

To prevent attackers from using CipherScan's server to probe internal infrastructure (e.g., calling `http://127.0.0.1:8080` or cloud metadata endpoints `http://169.254.169.254`), CipherScan performs DNS lookups and verifies that destination IP addresses are strictly public and routable.

C Concurrent Threat Processing

The server fires parallel asynchronous workers:

- `sandboxService.ts`: Playwright headless Chromium traces redirects and takes a screenshot.
- `reputationService.ts`: Queries VirusTotal v3 and Google Safe Browsing v4.
- `indianFraudEngine.ts`: Evaluates UPI VPA parameters and banking typosquatting.

D Database Insertion & Response Serialization

The combined verdict, redirect history, risk score, and Base64 screenshot are inserted into the PostgreSQL `scans` table and returned to the caller within < 500ms.

2. SSRF Protection Architecture in `urlSafety.ts`

```
// Comprehensive IP Range Exclusion in urlSafety.ts
const BLOCKED_IP_RANGES = [
  /^127\./, // IPv4 Loopback (127.0.0.0/8)
  /^10\./, // Private Network Class A (10.0.0.0/8)
  /^172\.(1[6-9]|2[0-9]|3[01])\./, // Private Network Class B (172.16.0.0/12)
  /^192\.168\./, // Private Network Class C (192.168.0.0/16)
  /^169\.254\./, // Link-Local / Cloud Metadata (169.254.0.0/16)
  /^0\./, // Broadcast / Zero address
  /^::1$/, // IPv6 Loopback
  /^[fF][cCdD]/ // IPv6 Unique Local Address
];
```

6. Sandboxing, Heuristics & Indian FinTech Sentinel

1. Playwright Headless Chromium Sandboxing

In `sandboxService.ts`, Playwright opens a clean, sandboxed Chromium instance for each scan:

- **Redirect Trace Array:** It registers an HTTP response listener that records every intermediate status code (`301 Moved Permanently`, `302 Found`, `307 Temporary Redirect`) and logs the URL chain into a JSON array:

```
[
  { "url": "https://bit.ly/3xFraud", "status": 301 },
  { "url": "https://redirect-hop.net/track", "status": 302 },
  { "url": "https://sbi-fake-portal.xyz/login", "status": 200 }
]
```

- **Visual Evidence Screenshot:** Takes a full-viewport JPEG capture at 1280×800 , compresses it into a lightweight buffer, and encodes it into a standard Base64 Data URI (`data:image/jpeg;base64, ...`).
- **Metadata Extraction:** Scrapes the document `<title>`, favicon, and canonical meta tags to identify deceptive brand logos.

2. Indian FinTech & UPI Sentinel (`indianFraudEngine.ts`)

The Unified Payments Interface (UPI) is the dominant digital payment mechanism in India, making it a prime target for cybercriminals. Attackers abuse standard UPI URIs: `upi://pay?pa=...&pn=...&am=...`

Threat 1: The "Reverse-Debit" Cashback Scam

A fraudster calls a victim claiming to send a ₹5,000 lottery or cashback prize. They send a link formatted as `upi://pay?pa=scammer@okhdfcbank&pn=CashbackDepartment&am=5000`. If the victim taps it, their UPI app opens and prompts for their secret UPI PIN. **Entering a PIN ALWAYS debits money, never receives money.**

CipherScan's Detection: The engine parses the `am=` parameter. If an amount is specified alongside promotional/refund keywords, it immediately raises a **CRITICAL 95 Risk Alert**: *"Debit Trap Detected: Entering your UPI PIN will DEDUCT ₹5,000 from your bank account!"*

Threat 2: VPA Impersonation & Free Cloud Typosquatting

Scammers register consumer handles like `sbi.support@paytm` or host phishing clones on free cloud services like `sbi-netbanking.vercel.app`.

CipherScan's Detection: Uses a **Levenshtein Distance algorithm** comparing the target domain against a dictionary of trusted Indian banking domains (SBI, HDFC, ICICI, Axis, PNB, Kotak, IRCTC, Mahadiscom). If a domain has a small edit distance or embeds bank names inside free-tier cloud subdomains, it is immediately flagged as a high-risk brand clone.

3. Section 65B Cybercrime Evidence Dossier

Under Section 65B of the Indian Evidence Act, digital evidence must maintain an unbroken chain of custody with exact timestamps and technical indicators. CipherScan auto-generates a structured forensic report containing:

- Exact UTC and IST scan timestamps.
- Complete unrolled redirect chain with intermediate IP addresses.
- Cryptographic SHA-256 hash of the webpage screenshot.
- Ready-to-submit summary for filing complaints on **cybercrime.gov.in** or dialing the **1930 Helpline**.

7. Database & Persistence Layer (`lib/db/`)

CipherScan utilizes **Neon Serverless PostgreSQL** paired with **Drizzle ORM**. Unlike traditional heavy ORMs, Drizzle provides zero-overhead, type-safe SQL queries with instant cold-start times.

Database Schema: `scansTable` (`lib/db/src/schema/scans.ts`)

Column Name	PostgreSQL Type	Description & Purpose
<code>id</code>	<code>serial PRIMARY KEY</code>	Unique incremental identifier for each scan record.
<code>originalUrl</code>	<code>text NOT NULL</code>	The exact link string tapped or submitted by the user.
<code>finalUrl</code>	<code>text</code>	The resolved destination URL after all HTTP redirects are traversed.
<code>isSafe</code>	<code>boolean NOT NULL</code>	Quick Boolean indicator (<code>true</code> = Safe, <code>false</code> = Threat).
<code>verdict</code>	<code>text NOT NULL</code>	Enum string: <code>"safe"</code> , <code>"suspicious"</code> , or <code>"malicious"</code> .
<code>riskScore</code>	<code>integer NOT NULL</code>	Calculated unified threat score (0 to 100).
<code>threatCategory</code>	<code>text</code>	Classification: e.g., <i>Phishing</i> , <i>Malware</i> , <i>UPI Fraud</i> , <i>Smishing</i> .
<code>redirectChain</code>	<code>jsonb</code>	Structured JSON array of all intermediate hop URLs and HTTP status codes.
<code>reasons</code>	<code>jsonb</code>	Array of human-readable bullet points explaining the score.
<code>previewImageUrl</code>	<code>text</code>	Data URI / Base64 encoded JPEG preview of the rendered page.
<code>triggerType</code>	<code>text NOT NULL</code>	Origin of scan: <code>"link"</code> (OS intent), <code>"camera"</code> (QR), or <code>"manual"</code> .
<code>virusTotalScore</code>	<code>integer</code>	Number of security vendors on VirusTotal flagging the URL as malicious.
<code>googleSafeBrowsing</code>	<code>boolean</code>	Whether Google's Threat Feeds flagged the URL.
<code>createdAt</code>	<code>timestamp</code>	Timestamp of scan creation (defaults to <code>NOW()</code>).

8. Web Dashboard & BFF Security Architecture (`artifacts/cipherscan/``)

The Backend-For-Frontend (BFF) Security Pattern

The Fundamental Security Flaw in Client-Side SPAs

Single Page Applications (React, Vue) run entirely inside the end-user's browser. If the React dashboard made requests directly to the Render backend, the client bundle would need to hold the master `APP_API_KEY`. Any user could open Chrome DevTools (F12) → Network tab, extract the API key, and flood or exhaust the backend.

How CipherScan Solves This via Vercel Serverless BFF (`api/proxy.js`):

1. The React application makes relative requests to its own domain: `GET /api/scans` or `POST /api/analyze`.
2. A Vercel Serverless Function (`proxy.js`) intercepts the request on the server side.
3. The serverless function attaches the confidential `APP_API_KEY` header securely server-to-server.
4. It proxies the request to the Render API server and streams back the result.
5. **The secret API key is 100% hidden from the public client browser.**

Key Web Application Views

- `Dashboard.tsx`: The central security operations cockpit. Displays real-time KPIs (Total Scans, Blocked Threats, Safe Links, Average Risk Score), 7-day volumetric timeline charts, and threat category distribution.
- `Analyze.tsx`: Interactive manual submission suite allowing security analysts to inspect arbitrary web links or UPI payment strings.
- `ScanHistory.tsx`: Forensic scan archive with real-time text searching, verdict filtering (All, Safe, Suspicious, Malicious), and expandable redirect chain views.
- `ThreatIntel.tsx`: Live radar tracking newly detected zero-day domains and malicious VPA handles.

9. Monorepo Contracts & Code Generation (`lib/``)

In large software systems, backend APIs and frontend UIs frequently break when response fields are renamed or removed. CipherScan eliminates this using a **Contract-First Architecture**:

1 OpenAPI Specification (`lib/api-spec/``)

The single source of truth defining every route, parameter, status code, and JSON structure in standard OpenAPI 3.1 YAML format.

2 Runtime Zod Schemas (`lib/api-zod/`)

Generated from the OpenAPI spec. Enforces strict input validation on the Express backend. If a request contains invalid types, Zod rejects it before reaching controller logic.

3 Typed React Query Hooks (`lib/api-client-react/`)

Generated via Orval. Provides fully typed React hooks (e.g., `useGetDashboardStats()` , `useAnalyzeUrl()`). If an API contract changes, TypeScript will flag compile errors across the frontend immediately.

10. Threat & Limitation Matrix (Where It Works vs. Fails)

Real-World Scenario	Protection Status	Technical Explanation
SMS Smishing Links	100% PROTECTED	Android OS triggers standard ACTION_VIEW intent; CipherScan intercepts before any browser executes code.
WhatsApp / Telegram Links	100% PROTECTED	Clicking links in messaging apps routes through the OS browser chooser to CipherScan.
Camera QR Code Scans	100% PROTECTED	Native camera QR decoders broadcast standard system URL view intents.
UPI Reverse-Debit Links	100% PROTECTED	Intercepted by dedicated uri: //pay intent filter; parses am= and VPA handles.
Multi-Hop URL Shorteners	100% PROTECTED	Playwright traces all 301/302 hops until terminal landing page is unmasked.
In-App Social Media Browsers (Instagram, TikTok)	CANNOT AUTO-INTERCEPT	Social media apps open links inside their own private, embedded WebView without notifying Android. (User must tap "Open in Chrome" for CipherScan to catch it).
SSRF / Private IP Probing (192.168.1.1)	INTENTIONALLY BLOCKED	Blocked by urlSafety.ts pre-flight checks to prevent internal network scanning.
Offline / Airplane Mode	REQUIRES NETWORK	Cloud reputation checks and sandboxing require an active internet connection.

11. Viva, Project Evaluation & Technical Defense Guide

Q1: "Why build an OS-level interceptor instead of a browser extension?"

Ideal Answer: Over 75% of malicious links and phishing attacks occur on mobile devices through SMS, WhatsApp, and QR codes — where desktop browser extensions do not exist. By implementing Android Intent Interception with ACTION_VIEW and BROWSABLE filters, CipherScan protects the user across every app on the device before any browser can launch.

Q2: "How do you protect your cloud sandbox against Server-Side Request Forgery (SSRF)?"

Ideal Answer: In `urlSafety.ts`, we implemented strict pre-flight DNS and IP validation. Before the server contacts any address, it resolves DNS and blocks private LAN ranges (RFC 1918), loopback addresses (`127.0.0.1`), link-local IPs, and cloud metadata endpoints (`169.254.169.254`). Only public HTTP, HTTPS, and UPI protocols are permitted.

Q3: "How does your system prevent secret API key theft from the web frontend?"

Ideal Answer: We implemented a Backend-For-Frontend (BFF) serverless architecture. The React frontend makes relative same-origin calls to `/api/*` on Vercel without holding any API keys. A Vercel Serverless Function (`proxy.js`) injects the secret `APP_API_KEY` server-to-server before forwarding the request to Render, ensuring client DevTools never expose secrets.

Q4: "What makes your UPI fraud detection unique compared to commercial antivirus tools?"

Ideal Answer: Global antivirus vendors focus on Western threat vectors (desktop PE binaries, Windows malware) and completely ignore Indian and emerging market FinTech threats. CipherScan treats UPI payment deep-links (`upi://pay`), parameter tampering, and VPA impersonation as first-class citizens alongside traditional web URLs.